

When things go wrong

SPRFST separates two different ideas: a value that might be missing, and an operation that might fail.

Might be missing: `T?`

A question mark means "or nothing".

```
use std.io

fn main() {
    let ages = ["ada": 36]
    let found = ages.get("ada")      ~~ this is Int?
    let missing = ages.get("nobody")

    io.say("{found ?? 0}")
    io.say("{missing ?? 0}")
}
```

`??` supplies a value for the nothing case. The compiler will not let you use an `Int?` where an `Int` is wanted, so a missing value can never slip through unnoticed.

`?.` calls a method only when there is something there:

```
use std.io

fn first_word(line: Text?) -> Text {
    give line?.split(" ").first() ?? "nothing"
}

fn main() {
    io.say(first_word("hello there"))
    io.say(first_word(nil))
}
```

Might fail: `Result<T>`

A function that can fail gives back `Ok(value)` or `Err(reason)`.

```
use std.io

fn parse_age(text: Text) -> Result<Int> {
    let n = text.to_int()
    if n == nil { give Err("'{}' is not a number") }
    let age = n ?? 0
    if age < 0 { give Err("an age cannot be negative") }
    give Ok(age)
}

fn main() {
    for input in ["34", "abc", "-2"] {
```

```

        match parse_age(input) {
            when Ok(age) -> io.say("{input}: age {age}")
            when Err(why) -> io.say("{input}: {why}")
        }
    }
}

```

Stopping on the spot

Sometimes the right answer is "this should never happen". `fail` raises a problem that travels up until something catches it.

```

use std.io

fn divide(a: Int, b: Int) -> Num {
    if b == 0 { fail "divide by zero" }
    give a / b
}

fn main() {
    io.say("{divide(10, 2)}")
    try {
        io.say("{divide(1, 0)}")
    } catch problem {
        io.say("caught: {problem}")
    }
    io.say("carrying on")
}

```

Use `Err` for problems the caller should think about, and `fail` for mistakes in the program itself.

Cleaning up with `defer`

`defer` runs when the function leaves, whichever way it leaves.

```

use std.io

fn work() {
    defer io.say(" closed")
    io.say(" opened")
    io.say(" working")
}

fn main() {
    io.say("start")
    work()
    io.say("done")
}

```

Exercise

Write `safe_sqrt(n)` that gives `Err` for negative input, then use it on a list of numbers and print one line per result.